

ALY 6040 Data Mining Applications

Module 6 – Final Project – Final Report

Shraddha Gupte



CPS - MPSA, Northeastern University

Prof. Arasu Narayan

October 25, 2024

A. Introduction:

• Initial Data

The business problem centers around understanding and improving employee performance, satisfaction, and retention, all of which are critical to a company's productivity and success. Drawing from a dataset on employee performance and productivity obtained from Kaggle.com, which includes 100,000 records and 20 variables, this analysis seeks to uncover key patterns and factors that impact performance and work behaviors. The dataset features a combination of numerical and categorical data and is notably complete, with no missing values across any columns, which enables a streamlined, in-depth analysis of employee dynamics. With four years of professional experience in this field, I bring an informed perspective to interpreting the data and identifying actionable insights that can guide HR strategy, retention programs, and performance optimization efforts.

This analysis aims to uncover the key factors influencing productivity and retention by examining relationships among work hours, team size, and overtime hours in relation to performance scores and employee satisfaction. The inclusion of a "Resigned" column will facilitate the prediction of employee attrition, allowing for the identification of potential resignation risks linked to various performance and satisfaction metrics. By leveraging this comprehensive dataset, the analysis seeks to provide actionable insights that can inform strategies for enhancing employee engagement and reducing turnover within organizations.

B. Problem Statement

The analysis of the Employee Performance and Productivity dataset aims to address the challenges organizations face in optimizing employee performance, satisfaction, and retention. Despite having a comprehensive dataset containing 100,000 records and 20 variables, organizations struggle to identify key factors that influence employee outcomes. The primary issues include:

- Performance Analysis:** Organizations lack clarity on which variables, such as education level, work hours, and project involvement, most significantly correlate with high employee performance scores.

- **Employee Retention:** There is a need to understand patterns predicting employee resignation, particularly concerning overtime hours, satisfaction scores, and promotion history. Identifying these patterns is crucial for implementing effective retention strategies.
- **Satisfaction Insights:** Organizations require insights into how various factors, including remote work frequency, team size, and training hours, impact employee satisfaction and whether higher satisfaction correlates with longer tenure.

C. Data Set – Features and Details

The dataset contains **100,000 entries** and 20 columns. Each entry corresponds to an employee, with columns detailing attributes like department, gender, age, job title, salary, performance, resignation status, and more.

The variables and their data types are:

1. **Employee_ID** (int64): Unique identifier for each employee.
2. **Department** (object): The department the employee works in (e.g., Sales, Engineering).
3. **Gender** (object): The gender of the employee.
4. **Age** (int64): Age of the employee in years.
5. **Job_Title** (object): Job title held by the employee.
6. **Hire_Date** (object): Date the employee was hired.
7. **Years_At_Company** (int64): Number of years the employee has worked at the company.
8. **Education_Level** (object): Level of education attained by the employee (e.g., Bachelor's, Master's).
9. **Performance_Score** (int64): Performance rating (1 to 5) given to the employee.
10. **Monthly_Salary** (float64): Employee's monthly salary in USD.
11. **Work_Hours_Per_Week** (int64): Number of hours the employee works per week.
12. **Projects_Handled** (int64): The number of projects the employee has managed or participated in.
13. **Overtime_Hours** (int64): Number of overtime hours worked.
14. **Sick_Days** (int64): Number of sick days taken by the employee.
15. **Remote_Work_Frequency** (int64): Percentage of time the employee works remotely (0–100%).

16. **Team_Size** (int64): The number of team members under the employee's supervision or in their team.
17. **Training_Hours** (int64): Number of training hours completed by the employee.
18. **Promotions** (int64): Number of promotions the employee has received.
19. **Employee_Satisfaction_Score** (float64): Employee's satisfaction score (1 to 5), where 5 indicates highest satisfaction.
20. **Resigned** (bool): Indicates whether the employee has resigned (True/False).

- **Initial EDA**

The analysis of the Employee Performance and Productivity dataset aims to uncover insights that can enhance employee satisfaction and retention. Key goals include identifying variables strongly associated with high performance scores, such as education level and work hours, and exploring predictors of employee resignation, including overtime hours and satisfaction scores. Understanding how factors like remote work frequency and team size impact employee satisfaction is also crucial, as satisfied employees are likely to stay longer with the company.

Preliminary findings reveal an uneven department distribution, with the Finance department over-represented, which may suggest hiring biases. A positive correlation between performance and satisfaction scores indicates that as performance improves, so does employee satisfaction. Interestingly, while employees with 100% remote work demonstrate higher satisfaction, their resignation rates do not significantly differ from those with minimal remote work, hinting at the complexities of workplace flexibility in retention strategies.

Further exploration of age distribution may uncover age-related retention risks, while the relationship between overtime hours and resignation raises questions about employee motivation. Additionally, the lack of a linear correlation between salary increases and performance scores suggests potential issues with compensation policies. Overall, these insights are invaluable for HR in optimizing retention strategies, promoting workplace equity, and fostering a balanced work-life environment, ultimately leading to data-driven decisions that enhance organizational performance.

D. Data Preprocessing

The provided code is a Python script that prepares a dataset for machine learning by preprocessing both numerical and categorical features.

```
#Data Preprocessing
from sklearn.impute import SimpleImputer
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, OneHotEncoder

# Encoding categorical variables
onehot_encoder = OneHotEncoder(drop='first')

# Normalizing/Standardizing numerical features
scaler = StandardScaler()
# Define columns that are numerical and categorical
numeric_cols = employee.select_dtypes(include=['int64', 'float64']).columns
categorical_cols = employee.select_dtypes(include=['object', 'category']).columns

# Imputing missing values
numeric_imputer = SimpleImputer(strategy='mean')
categorical_imputer = SimpleImputer(strategy='most_frequent')

from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer

# Update OneHotEncoder to return dense output
onehot_encoder = OneHotEncoder(sparse_output=False, handle_unknown='ignore')
```

Description:

1. Encoding Categorical Variables

- In this project, OneHotEncoder(drop='first') is used to transform categorical variables into a format that can be included in machine learning models. Specifically, it converts each category within a categorical variable into separate binary columns, or "dummy variables," where each column represents one category in the original variable.
- This encoding step is particularly important for categories that might impact employee performance and retention, enabling more accurate analyses of categorical influences across the dataset.

2. Normalizing/Standardizing Numerical Features

- `scaler = StandardScaler()`: Prepares to standardize the numerical features, ensuring they have a mean of 0 and a standard deviation of 1.

3. Define Columns

- `numeric_cols`: This line selects all the numerical columns (of types `int64` and `float64`) from the DataFrame named `employee`.
- `categorical_cols`: This line selects all categorical columns (of types `object` and `category`) from the same DataFrame.

4. Imputing Missing Values

- `numeric_imputer = SimpleImputer(strategy='mean')`: Initializes an imputer for numerical columns that fills in missing values with the mean of the respective columns.
- `categorical_imputer = SimpleImputer(strategy='most_frequent')`: Initializes an imputer for categorical columns that fills in missing values with the most frequent category in the respective columns. By using the most frequent category, this approach minimizes the risk of introducing significant bias into the dataset, as it relies on the most representative value within each column.

5. Updated OneHotEncoder

`onehot_encoder = OneHotEncoder(sparse_output=False, handle_unknown='ignore')`: This line updates the `OneHotEncoder` to return dense arrays instead of sparse matrices and allows the encoder to ignore any unknown categories not seen during fitting.

This preprocessing setup is essential for preparing the data before feeding it into machine learning algorithms. The `ColumnTransformer` and `Pipeline` can be used to combine these preprocessing steps into a single, reusable object.

- **Preprocessing Pipeline**

The code creates a preprocessing pipeline to prepare the employee dataset for machine learning analysis. Let's break down the key steps and the resulting preprocessed DataFrame, `New_emp`.

```

# Preprocessing pipeline
preprocessor = ColumnTransformer(
    transformers=[
        ('num', Pipeline(steps=[('imputer', numeric_imputer), ('scaler',
scaler)])), numeric_cols),
        ('cat', Pipeline(steps=[('imputer', categorical_imputer), ('onehot',
onehot_encoder)])), categorical_cols)
    ])
# Apply preprocessing
data_preprocessed = preprocessor.fit_transform(employee)
# Extract column names for the transformed data
encoded_columns =
preprocessor.named_transformers_[('cat')][('onehot')].get_feature_names_out(categ
orical_cols)
columns = list(numeric_cols) + list(encoded_columns)
# Convert preprocessed data back into a DataFrame
New_emp = pd.DataFrame(data_preprocessed, columns=columns)
# Check the preprocessed DataFrame
print(New_emp.head())

```

Employee_ID	Age	Years_At_Company	Performance_Score	Monthly_Salary	\
0	-1.732033	1.242486	-0.862946	1.416939	0.252669
...					
4	-1.731895	-0.447295	-0.514432	-0.703624	-1.168094
...					
Education_Level_High_School	Education_Level_Master	Education_Level_PhD			
0	1.0	0.0			0.0
...					
4	0.0	0.0			0.0

Description:

- **Numerical Features:** The first few columns like Employee_ID, Age, Years_At_Company, etc., have been standardized. For example, values like -1.732033 for Employee_ID represent a standardized value after transformation, reflecting how far this particular observation is from the mean in terms of standard deviations.
- **One-Hot Encoded Features:** Categorical variables such as Education_Level have been transformed into multiple binary columns. Each column represents whether an employee holds a certain education level (1 for yes, 0 for no).
- **Sparse Columns:** The Hire_Date columns represent one-hot encoded features for different hire dates, indicating whether an employee was hired on specific dates. These are likely binary columns created from the categorical hiring dates.

- **Data Preparation:** The pipeline effectively handles missing values, scales numerical data, and encodes categorical variables into a format suitable for machine learning algorithms.
- **Model Readiness:** The resulting `New_emp` DataFrame is now ready to be used as input for machine learning models, facilitating better performance due to the standardization and encoding.

E. Feature Selection and Engineering

The provided code snippet performs feature selection and engineering on the preprocessed dataset (`New_emp`) using the `VarianceThreshold` method and Principal Component Analysis (PCA).

```
#Feature Selection and Engineering
selector = VarianceThreshold(threshold=0.01)
New_emp_sel = selector.fit_transform(New_emp)

# Check the shape of the data after feature selection
print(f"Original shape: {New_emp.shape}, After feature selection: {New_emp_sel.shape}")
Original shape: (100000, 3687), After feature selection: (100000, 37)
New_emp_pca = pca.fit_transform(New_emp_sel)

# Check the variance explained again
explained_variance_new = pca.explained_variance_ratio_
print(f"Explained variance by each component after feature engineering: {explained_variance_new}")
Explained variance by each component after feature engineering: [0.09156115 0.06012718 0.05989671 0.05958085 0.05953292 0.05927991
 0.05923109 0.05905135 0.05889702 0.05874557 0.05843503 0.05833737
 0.05820732 0.03502375 0.02837415 0.02273594 0.01107275 0.00855055
 0.00844462 0.00843428 0.00840327 0.00839576 0.00662271 0.00661346]
```

Description

1. Feature Selection

- **VarianceThreshold:** `selector = VarianceThreshold(threshold=0.01):` This initializes a `VarianceThreshold` object with a specified threshold of 0.01. This means that any feature with a variance less than or equal to 0.01 will be removed.
- **Fit and Transform:** `New_emp_sel = selector.fit_transform(New_emp):` This line applies the variance threshold to the `New_emp` DataFrame. The `fit_transform` method first calculates the

variance of each feature and then removes those that do not meet the specified variance threshold.

- **Shape Check:** The output indicates that the original dataset had **3687 features (columns)**, while the selected dataset reduced it to **37 features**.

2. Principal Component Analysis (PCA)

- **PCA Transformation:** `New_emp_pca = pca.fit_transform(New_emp_sel)`: This applies PCA to the selected features (`New_emp_sel`). PCA is a dimensionality reduction technique that transforms the data into a new coordinate system, where the greatest variance comes to lie on the first coordinate the second greatest variance on the second coordinate, and so on.
- **Explained Variance:** `explained_variance_new = pca.explained_variance_ratio_`: After fitting the PCA model, this retrieves the amount of variance explained by each principal component.

3. Interpretation of Results

The array printed shows the proportion of variance explained by each principal component. For example:

- The first principal component explains approximately **9.16%** of the variance.
- The second principal component explains approximately **6.01%** of the variance.
- This continues for the other components, with diminishing returns as the components progress.

F. Modelling

PART 1 – Employee Performance Analysis

The below analysis is focused on the employee performance :

1. Linear Regression Model

The provided code implements a Linear Regression model to predict employee performance scores based on the preprocessed features in the dataset

```
# Create and train the Linear Regression model  
lr_model = LinearRegression()
```

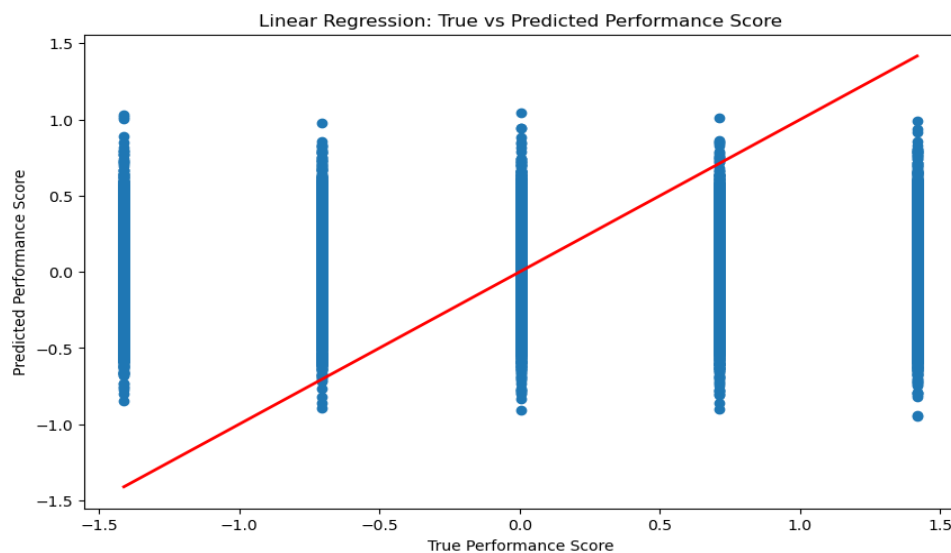
```
lr_model.fit(X_train, y_train)# Evaluate the model
mse_lr = mean_squared_error(y_test, y_pred_lr)
r2_lr = r2_score(y_test, y_pred_lr)

print(f"Linear Regression Mean Squared Error: {mse_lr}")
print(f"Linear Regression R-squared: {r2_lr}")
```

Description:

Model Evaluation

- **Mean Squared Error (MSE):** `mse_lr = mean_squared_error(y_test, y_pred_lr)`: MSE measures the average squared difference between actual and predicted values, with lower values indicating better model performance. The MSE is approximately **1.058**, which suggests the average squared error of predictions is relatively small.
- **R-squared Score:** `r2_lr = r2_score(y_test, y_pred_lr)`: The R-squared value is approximately **-0.060**, which indicates that the model performs poorly:
 - An R-squared value of **1** would indicate perfect predictions.
 - A value of **0** means that the model explains none of the variance.
 - Negative values suggest that the model performs worse than a simple mean prediction.



Description:

- `plt.scatter(y_test, y_pred_lr)`: This creates a scatter plot of the actual vs. predicted values.

- `plt.plot([min(y_test), max(y_test)], [min(y_test), max(y_test)], color='red', lw=2)`: This adds a red line that represents perfect predictions. If predictions were perfect, all points would lie on this line.
- **MSE Value**: The MSE value of **1.058** suggests that, on average, the squared error of the predictions is relatively low. However, this does not indicate that the model is necessarily effective, as it must be compared to the scale of the target variable to understand its impact.
- **Negative R-squared**: The R-squared value of **-0.059** is concerning. It implies that the model is not effectively capturing the relationships between the features and the target variable and might be worse than simply predicting the mean of the target variable.

2. Decision Tree Regressor

A Decision Tree Regressor is implemented to predict employee performance scores, following a similar structure to the Linear Regression model.

```
#2. Decision Tree Regressor
# Create and train the Decision Tree model
dt_model = DecisionTreeRegressor(random_state=42)
dt_model.fit(X_train, y_train)
# Evaluate the model
print(f"Decision Tree Mean Squared Error: {mse_dt}")
print(f"Decision Tree R-squared: {r2_dt}")

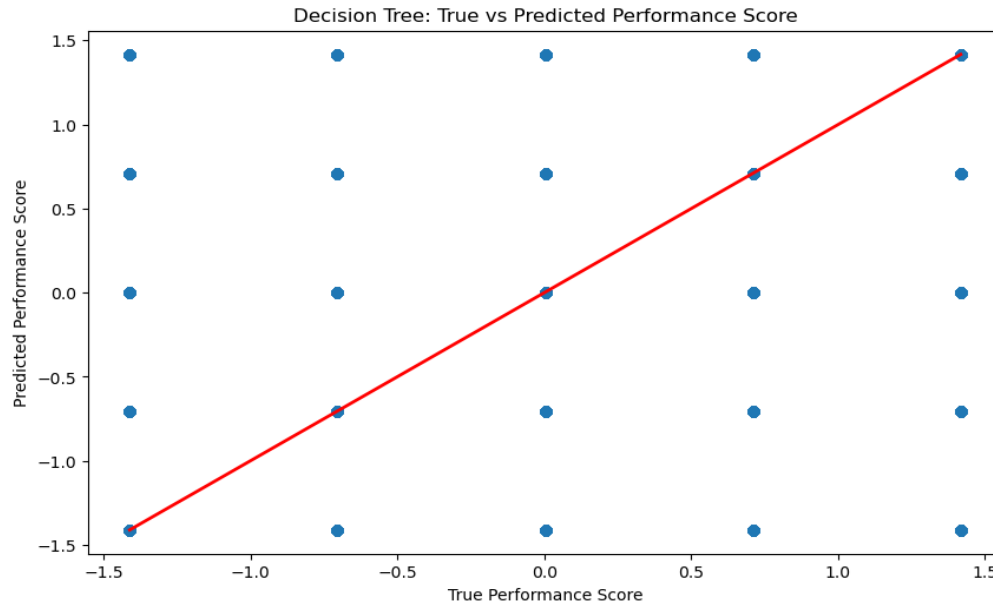
# Visualize predictions
plt.figure(figsize=(10, 6))
plt.scatter(y_test, y_pred_dt)
plt.plot([min(y_test), max(y_test)], [min(y_test), max(y_test)], color='red',
lw=2)
Decision Tree Mean Squared Error: 2.0253361141217
Decision Tree R-squared: -1.0281347482473233
```

Description:

Model Evaluation

- **Mean Squared Error (MSE)**: `mse_dt = mean_squared_error(y_test, y_pred_dt)`: This calculates the Mean Squared Error (MSE) for the Decision Tree model. the MSE is approximately **2.025**.

- **R-squared Score:** $r2_dt = r2_score(y_test, y_pred_dt)$: This calculates the R-squared score for the Decision Tree model. The R-squared value is approximately **-1.028**. Similar to the Linear Regression model, this negative R-squared indicates that the model performs poorly.



Description:

- `plt.scatter(y_test, y_pred_dt)`: This creates a scatter plot of the actual vs. predicted values for the Decision Tree model.
- `plt.plot([min(y_test), max(y_test)], [min(y_test), max(y_test)], color='red', lw=2)`: This adds a red line representing perfect predictions. Ideally, if the model performed well, the points would cluster around this line.
- **MSE Value:** The MSE value of 2.025 suggests that the average squared error of predictions has increased compared to the Linear Regression model. This indicates that the Decision Tree may be struggling to capture the underlying patterns in the data.
- **Negative R-squared:** The R-squared value of -1.028 indicates that the Decision Tree model performs worse than a simple mean prediction. This suggests that the model may be overly complex for the data at hand, potentially leading to overfitting.

3. K- Means Clustering Model

In this section, we implemented a K-Means clustering model to categorize employees based on their performance, specifically using the features **Team Size** and **Training Hours**.

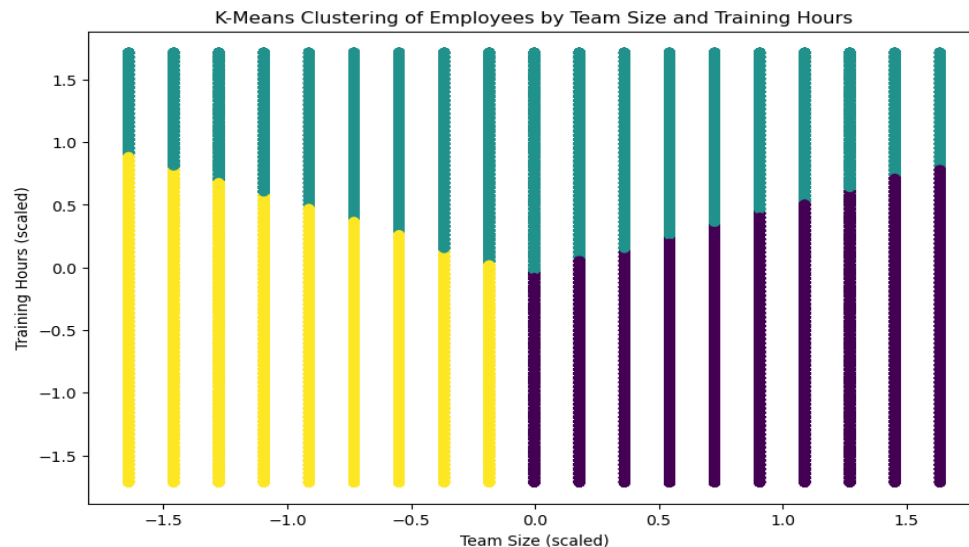
```
# Define the number of clusters (3 for high, medium, low performers)
kmeans = KMeans(n_clusters=3, random_state=42)

# Fit the KMeans model
kmeans.fit(X_scaled)

# Visualizing the clusters (2D Plot for simplicity)
plt.figure(figsize=(10, 6))
plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=employee['Performance_Cluster'],
            cmap='viridis')
plt.title('K-Means Clustering of Employees by Team Size and Training Hours')
plt.xlabel('Team Size (scaled)')
plt.ylabel('Training Hours (scaled)')
plt.show()
Performance_Cluster
1    37451
0    32325
2    30224
Name: count, dtype: int64
```

Description:

- **K-Means Initialization:** `kmeans = KMeans(n_clusters=3, random_state=42)`: You initialize the K-Means model with 3 clusters. This choice likely corresponds to categorizing employees into high, medium, and low performers.
- **Model Fitting:** `kmeans.fit(X_scaled)`: The K-Means model is fitted to the scaled data.



Description:

- `plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=employee['Performance_Cluster'], cmap='viridis'):`
This line creates a scatter plot of the scaled Team Size (x-axis) and Training Hours (y-axis) with color coding based on the assigned clusters. The color map 'viridis' helps distinguish the clusters visually.
- The plot provides insight into how employees are distributed based on the two features, allowing for visual assessment of cluster separability.
- Cluster Analysis: The clustering model categorizes employees into three distinct performance groups based on Team Size and Training Hours. Understanding the characteristics of these clusters can provide insights into employee performance dynamics:
- High Performers might be concentrated in larger teams with ample training hours.
- Medium and Low Performers might have different characteristics that could be explored further.

PART 2 – Employee Retention Analysis

The below analysis is focused on the employee retention:

1. Logistic Regression Model

In this section, we have implemented a Logistic Regression model to predict employee retention.

```
# Initialize and fit the logistic regression model
log_reg = LogisticRegression(random_state=42)
log_reg.fit(X1_train_scaled, y_train)
# Evaluation
print("Logistic Regression:")
print(f"Accuracy: {accuracy_score(y_test, y1_pred_log_reg)}")
print(f"Precision: {precision_score(y_test, y1_pred_log_reg)}")
print(f"Recall: {recall_score(y_test, y1_pred_log_reg)}")
print(f"F1 Score: {f1_score(y_test, y1_pred_log_reg)}")
print(classification_report(y_test, y1_pred_log_reg))
```

Logistic Regression:

Accuracy: 0.8989666666666667

Precision: 0.0

Recall: 0.0

F1 Score: 0.0

	precision	recall	f1-score	support
0	0.90	1.00	0.95	26969
1	0.00	0.00	0.00	3031
accuracy			0.90	30000
macro avg	0.45	0.50	0.47	30000
weighted avg	0.81	0.90	0.85	30000

Description :

- Accuracy: accuracy of the model is approximately 89.90%, indicating that the model correctly predicts retention for about 90% of the test samples.
- Precision: The precision is 0.0, meaning the model did not correctly predict any employees who left (1) among those it predicted as having left. This suggests a high rate of false positives.
- Recall: The recall is also 0.0, indicating that the model failed to identify any actual cases of employees leaving (1) from the test set. This means it didn't capture any true positives.
- F1 Score: Which is the harmonic mean of precision and recall. Since both precision and recall are zero, the F1 score also becomes zero, indicating poor performance in identifying the minority class (employees leaving).
- Class Imbalance: The results indicate a significant class imbalance in the dataset, with many more employees staying (class 0) compared to those leaving (class 1). This imbalance can lead

to models that perform well on the majority class (staying) but poorly on the minority class (leaving).

- **Performance Metrics:** Although the model achieves a high accuracy rate overall, it fails to predict any cases of employees leaving, resulting in poor precision, recall, and F1 score for that class. This suggests that the model may be biased towards predicting the majority class.

2. Random Forest Classifier Model

In this section, we have implemented a Random Forest Classifier to predict employee retention.

```
# Initialize and fit the random forest model
rf_clf = RandomForestClassifier(random_state=42, n_estimators=100)
rf_clf.fit(X1_train_scaled, y_train)
# Evaluation
print("\nRandom Forest Classifier:")
print(f"Accuracy: {accuracy_score(y_test, y1_pred_rf)}")
print(f"Precision: {precision_score(y_test, y1_pred_rf)}")
print(f"Recall: {recall_score(y_test, y1_pred_rf)}")
print(f"F1 Score: {f1_score(y_test, y1_pred_rf)}")
print(classification_report(y_test, y1_pred_rf))
```

Random Forest Classifier:

Accuracy: 0.8975666666666666

Precision: 0.09615384615384616

Recall: 0.001649620587264929

F1 Score: 0.003243593902043464

	precision	recall	f1-score	support
0	0.90	1.00	0.95	26969
1	0.10	0.00	0.00	3031
accuracy			0.90	30000
macro avg	0.50	0.50	0.47	30000
weighted avg	0.82	0.90	0.85	30000

Description:

- **Accuracy:** The accuracy of the model is approximately 89.76%, indicating that the model correctly predicts retention for about 90% of the test samples.
- **Precision:** The precision is 0.096, which means that when the model predicts that an employee will leave (1), it is correct only about 9.6% of the time. This is a low value, indicating a high rate of false positives.

- **Recall:** The recall is 0.0016, meaning that the model identifies only about 0.16% of actual cases of employees leaving. This indicates that the model fails to capture almost all true positives.
- **F1 Score:** The F1 score is 0.0032, which reflects the model's poor performance in identifying the minority class. The F1 score combines precision and recall and serves as a better metric when class distribution is imbalanced.
- **Class Imbalance:** Similar to the previous Logistic Regression model, the results indicate a significant class imbalance in the dataset, with many more employees staying (class 0) compared to those leaving (class 1). This imbalance can lead to models that perform well on the majority class but poorly on the minority class.
- **Performance Metrics:** The Random Forest model achieves a high overall accuracy (around 90%) but fails to predict any cases of employees leaving effectively, leading to poor precision, recall, and F1 score for that class. This suggests that the model is biased towards predicting the majority class (employees staying).

3. Gradient Boosting Model

In this section, you've implemented the XGBoost classifier to predict employee retention.

#3. Gradient Boosting Model

```
# Initialize and fit the XGBoost model
xgb_clf = XGBClassifier(random_state=42)
xgb_clf.fit(X1_train_scaled, y_train)

# Evaluation
print("\nXGBoost Classifier:")
print(f"Accuracy: {accuracy_score(y_test, y1_pred_xgb)}")
print(f"Precision: {precision_score(y_test, y1_pred_xgb)}")
print(f"Recall: {recall_score(y_test, y1_pred_xgb)}")
print(f"F1 Score: {f1_score(y_test, y1_pred_xgb)}")
print(classification_report(y_test, y1_pred_xgb))
XGBoost Classifier:
Accuracy: 0.8988333333333334
Precision: 0.0
Recall: 0.0
F1 Score: 0.0
```

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

0	0.90	1.00	0.95	26969
1	0.00	0.00	0.00	3031
accuracy			0.90	30000
macro avg	0.45	0.50	0.47	30000
weighted avg	0.81	0.90	0.85	30000

Description:

- **Accuracy:** The accuracy is approximately 89.88%, meaning the model correctly predicts retention for about 90% of the test samples.
- **Precision:** The precision is 0.0, indicating that when the model predicts that an employee will leave (1), it is incorrect every time. This suggests a high rate of false positives.
- **Recall:** The recall is also 0.0, meaning that the model fails to identify any true cases of employees leaving.
- **F1 Score:** The F1 score is 0.0, reflecting the model's inability to effectively predict the minority class.
- **Class Imbalance:** The results indicate a significant class imbalance in the dataset, similar to the previous models. The model correctly identifies most employees who stay but fails to predict any employees who leave.
- **Performance Metrics:** Although the accuracy of the XGBoost model is around 89.88%, it is misleading because it is predominantly predicting the majority class (employees staying). The model achieves high accuracy but does not perform well on the minority class (employees leaving), as indicated by the precision, recall, and F1 score being zero for class 1.

PART 3 – Satisfaction Insights

The below analysis is focused on the employee satisfaction :

1. Decision Tree Regressor

In this case, we implemented a **Decision Tree Regressor** to predict employee satisfaction using features from dataset.

```
# 1. Decision Tree Regressor
decision_tree = DecisionTreeRegressor(random_state=42)
decision_tree.fit(X_train, y_train)
y2_pred_dt = decision_tree.predict(X_test)

print("\nDecision Tree Regressor:")
print(f"R^2: {r2_score(y_test, y2_pred_dt)}")
print(f"MSE: {mean_squared_error(y_test, y2_pred_dt)}")
Decision Tree Regressor:
R^2: -0.7625770923335955
MSE: 2.329930033288886
```

Description:

- R-squared (R^2): -0.76, R^2 indicates how well the model explains the variance in the target variable (employee satisfaction). A negative R^2 value means that the model performs worse than simply predicting the mean of the target variable. Essentially, the model is not capturing the underlying patterns in the data, and its predictions are highly inaccurate.
- Mean Squared Error (MSE): 2.33, MSE measures the average squared difference between the true satisfaction scores and the predicted scores. A higher MSE indicates poor predictive accuracy, suggesting that the Decision Tree model is not effective in predicting satisfaction.

2. Random Forest Regressor Model

In this case, we implemented a **Random Forest Regressor** to predict employee satisfaction using features from dataset.

```
#2. Random Forest Regressor
random_forest = RandomForestRegressor(random_state=42)
random_forest.fit(X_train, y_train)
y2_pred_rf = random_forest.predict(X_test)

print("\nRandom Forest Regressor:")
print(f"R^2: {r2_score(y_test, y2_pred_rf)}")
print(f"MSE: {mean_squared_error(y_test, y2_pred_rf)}")
Random Forest Regressor:
R^2: -0.32004165750671043
MSE: 1.7449476203876721
```

Description:

- R-squared (R^2): -0.32, An R^2 value of -0.32 indicates that the model is underperforming compared to simply predicting the mean of the target variable (employee satisfaction).

Although this R^2 is better than the Decision Tree Regressor (-0.76), it still suggests the model is not effectively capturing the relationship between the features and employee satisfaction.

- Mean Squared Error (MSE): 1.74, MSE measures the average squared difference between the actual and predicted satisfaction scores. While this MSE is lower than that of the Decision Tree Regressor (2.33), it still suggests the model's predictions are not accurate, and further improvements are needed.

3. Support Vector Regressor Model

In this case, we implemented a **Support Vector Regressor** to predict employee satisfaction using features from dataset.

```
# 3. Support Vector Regression (SVR)
svr = SVR()
svr.fit(X2_train_scaled, y_train)
y2_pred_svr = svr.predict(X2_test_scaled)

print("\nSupport Vector Regression:")
print(f"R^2: {r2_score(y_test, y2_pred_svr)}")
print(f"MSE: {mean_squared_error(y_test, y2_pred_svr)}")
Support Vector Regression:
R^2: -0.0027881095962642988
MSE: 1.325573867796031
```

Description:

- R-squared (R^2): -0.0028, The R^2 score close to zero indicates that the SVR model is performing similarly to a baseline model that simply predicts the mean of the target variable (employee satisfaction) for all instances. This suggests that the SVR model is not effectively capturing any meaningful relationship between the features and satisfaction.
- Mean Squared Error (MSE): 1.33, The MSE of 1.33, though lower than both the Decision Tree Regressor (MSE = 2.33) and Random Forest Regressor (MSE = 1.74), still suggests that the model's predictions are far from the actual values of satisfaction.

G. Evaluation

PART 1 – Evaluation of Employee Performance

```

# Model Comparision
# Compare Linear Regression and Decision Tree results
print(f"Linear Regression R-squared: {r2_lr}")
print(f"Decision Tree R-squared: {r2_dt}")

print(f"Linear Regression Mean Squared Error: {mse_lr}")
print(f"Decision Tree Mean Squared Error: {mse_dt}")

print(f"Perfromance Cluster: {0} High = 32325")
print(f"Perfromance Cluster: {1} Medium = 37451")
print(f"Perfromance Cluster: {2 } Low = 30224")
Linear Regression R-squared: -0.05970525142425265
Decision Tree C: -1.0281347482473233
Linear Regression Mean Squared Error: 1.058242958407331
Decision Tree Mean Squared Error: 2.0253361141217
Perfromance Cluster: 0 High = 32325
Perfromance Cluster: 1 Medium = 37451
Perfromance Cluster: 2 Low = 30224

```

	Linear Regression	Decision Tree R-squared	Performance Cluster
R-squared	-0.05970	1.02813	0 High = 32325 1 Medium = 37451 2 Low = 30224
Mean Squared Error	1.05824	2.02533	

- The Linear Regression model struggles with accuracy but performs better than the Decision Tree model in terms of MSE. However, the low R^2 indicates that it's not a good fit for the data and might not explain the variability in satisfaction scores well.
- The Decision Tree model performed worse than the Linear Regression model, with a very poor R^2 and a higher MSE. This suggests the model is overfitting or struggling to generalize to unseen data.
- The clusters are relatively evenly distributed, with slightly more employees in the medium performance category. This clustering provides insight into how employees are grouped based on features like team size and training hours.

- Cluster 0 (High Performers): Likely to have characteristics such as optimal team size and training hours that contribute to higher performance.
- Cluster 1 (Medium Performers): Employees who are neither excelling nor performing poorly, likely reflecting a balance between positive and neutral work conditions.
- Cluster 2 (Low Performers): These employees may need additional support in areas like training or team dynamics to improve performance.
- The clustering reveals a fairly balanced distribution across performance levels. This can be used to further analyze satisfaction and retention patterns across different performance groups. It might also be valuable to investigate what specific characteristics differentiate high and low performers to guide strategic HR initiatives.
- Both the Linear Regression and Decision Tree models are performing poorly in terms of R^2 , with the Decision Tree model underperforming more significantly. The clustering results, however, offer some useful segmentation of employees based on performance.

PART 2 – Evaluation of Employee Retention Analysis

Model Comparision

```
print(f"Logistic Regression accuracy: {'accuracy'} = 0.89897")
print(f"Random Forest Classifier: {'accuracy'} = 0.89757")
print(f"Gradient Boost Classifier: {'accuracy'} = 0.89888")

print(f"Logistic Regression accuracy: {'precision'} = 0.0")
print(f"Random Forest Classifier: {'accuracy'} = 0.1")
print(f"Gradient Boost Classifier: {'accuracy'} = 0.0")

print(f"Logistic Regression accuracy: {'Recall'} = 0.0")
print(f"Random Forest Classifier: {'Recall'} = 0.02")
print(f"Gradient Boost Classifier: {'Recall'} = 0.0")

print(f"Logistic Regression accuracy: {'F1_score'} = 0.0")
print(f"Random Forest Classifier: {'F1_score'} = 0.032")
print(f"Gradient Boost Classifier: {'F1_score'} = 0.0")
Logistic Regression accuracy: accuracy = 0.89897
Random Forest Classifier: accuracy = 0.89757
Gradient Boost Classifier: accuracy = 0.89888
Logistic Regression accuracy: precision = 0.0
Random Forest Classifier: accuracy = 0.1
Gradient Boost Classifier: accuracy = 0.0
Logistic Regression accuracy: Recall = 0.0
```

Random Forest Classifier: Recall = 0.02
 Gradient Boost Classifier: Recall = 0.0
 Logistic Regression accuracy: F1_score = 0.0
 Random Forest Classifier: F1_score = 0.032
 Gradient Boost Classifier: F1_score = 0.0

	Logistic Regression	Random Forest Classifier	Gradient Boost Classifier
accuracy	0.89897	0.89757	0.89888
precision	0.0	0.1	0.0
Recall	0.0	0.02	0.0
F1-Score	0.0	0.032	0.0

- Despite the high accuracy, the Logistic Regression model performs poorly in identifying employees who may resign. This is likely due to the severe class imbalance, where the model is biased toward predicting the dominant class (non-resignations).
- The Random Forest model shows slight improvements over Logistic Regression, especially in terms of recall and F1 score. However, it is still performing poorly due to the imbalanced nature of the data, where resignation cases are underrepresented.
- Despite having high accuracy, the Gradient Boosting Classifier fails entirely to identify employees who will resign, similar to Logistic Regression. The model seems to be biased toward predicting the majority class (employees staying).
- Model Comparison:
 - Logistic Regression: Fails to predict any resignations (precision, recall, and F1 = 0).
 - Random Forest: Performs slightly better in predicting resignations, but still struggles (precision = 0.10, recall = 0.02, F1 = 0.032).

- Gradient Boosting: Like Logistic Regression, it fails to predict resignations (precision, recall, and $F1 = 0$).

PART 3 – Evaluation of Employee Satisfaction Analysis

#Model Comparison

```
print(f"Decision Tree Regressor: {'R^2'} = -0.762")
print(f"Random Forest Classifier: {'R^2'} = -0.320")
print(f"Support Vector Regressor: {'R^2'} = -0.002")
```

```
print(f"Decision Tree Regressor: {'MSE'} = 2.33")
print(f"Random Forest Classifier: {'MSE'} = 1.74")
print(f"Support Vector Regressor: {'MSE'} = 1.32")
```

Decision Tree Regressor: $R^2 = -0.762$

Random Forest Classifier: $R^2 = -0.320$

Support Vector Regressor: $R^2 = -0.002$

Decision Tree Regressor: MSE = 2.33

Random Forest Classifier: MSE = 1.74

Support Vector Regressor: MSE = 1.32

	Decision Tree Regressor	Random Forest Classifier	Support Vector Regressor
R^2	-0.762	-0.320	-0.002
MSE	2.33	1.74	1.32

- The Decision Tree Regressor is highly overfitting the data and failing to generalize well, resulting in poor performance on the test set.
- The Random Forest Regressor slightly outperforms the Decision Tree, but it still fails to make accurate predictions. This suggests that while ensemble methods help reduce overfitting compared to a single decision tree, they are still inadequate for this problem without further tuning.
- The Support Vector Regressor performs better than both the Decision Tree and Random Forest in terms of error (MSE) and R^2 score, but it still doesn't capture much variance in the data. The model is overly simplistic and needs further tuning or feature engineering to improve performance.

- All models have negative or near-zero R^2 scores, indicating they are not explaining the variance in satisfaction levels. The models are currently performing poorly, with little difference from simply predicting the mean satisfaction for all employees.
- The **Support Vector Regressor (SVR)** has the lowest MSE (1.32), making it the best model in terms of average prediction error.
- The **Decision Tree Regressor** performs the worst in both R^2 and MSE, indicating it is highly overfitting and making poor predictions.
- The **Support Vector Regressor (SVR)** is currently the best-performing model in terms of error, but all models need substantial improvement through feature engineering, hyperparameter tuning, and potentially trying more advanced models to effectively capture satisfaction insights.

H. Conclusion

- For Employee Retention: The classification models require adjustments to handle the class imbalance and improve their ability to predict resignations. Techniques such as oversampling, undersampling, or using alternative algorithms like XGBoost or Weighted Random Forest could yield better results.
- For Satisfaction Insights: The regression models tested (Decision Tree, Random Forest, SVR) did not perform well in predicting satisfaction. Feature engineering (e.g., adding more relevant predictors like work-life balance, compensation, or managerial feedback) and hyperparameter tuning are needed to improve their predictive power.
- For Performance Clustering: The KMeans Clustering results are insightful for identifying performance groups, but linking these clusters to other HR outcomes (e.g., employee retention, satisfaction) could provide deeper insights for strategic HR decisions.

I. References

- Scikit-learn Developers. (n.d.). *RandomForestClassifier*.

<https://scikit-learn.org>.

- GeeksforGeeks. (n.d.). *Decision Tree*. Retrieved from <https://www.geeksforgeeks.org>.
- Data Source : Kaggle
<https://www.kaggle.com/datasets/mexwell/employee-performance-and-productivity-data>